Group ID: 2023-27/IT/SECTION/GID

A Project Report on

Smart Traffic Control System Using Vehicle Density Analysis

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology in Information Technology

Submitted by

Madhav Goel (Roll No.)
Akash Varshney (Roll No.)
Abhisekh Chaudhary (Roll No.)
Shivam Goel (Roll No.)

Under the Supervision of

Ruchi Gupta



Department of Information Technology
AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD
DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

October 27, 2026

Declaration

We hereby declare that the work presented in this report, entitled as **"Smart Traffic Control System Using Vehicle Density Analysis"**, was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors and sources for all the words, ideas, diagrams, graphics, computer programs, experiments, and results that are not our original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors and sources.

We have self-checked this report for plagiarism using appropriate tools and confirm that it meets the prescribed academic integrity standards.

Student Names	**Signatures**
Madhav Goel (Roll No.)	_____
Akash Varshney (Roll No.)	_____
Abhisekh Chaudhary (Roll No.)	_____
Shivam Goel (Roll No.)	_____

Date: October 27, 2026

Certificate

This is to certify that the project report entitled **"Smart Traffic Control System Using Vehicle Density Analysis"**, submitted by the following students in partial fulfillment of the requirements for the award of the degree of *Bachelor of Technology in Information Technology* to Ajay Kumar Garg Engineering College, Ghaziabad (affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow) is an authentic record of genuine work carried out by them under my supervision.

Students:
Madhav Goel (Roll No.) Akash Varshney (Roll No.) Abhisekh Chaudhary (Roll No.) Shivam Goel (Roll No.)

The matter presented in this report has not been submitted for the award of any other degree or diploma at this or any other institution.

Ruchi Gupta	**Dr. Rahul Sharma**
Supervisor	Head of Department
Department of Information Technology	Department of Information Technology
AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD	AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD

Date: October 27, 2026

Acknowledgements

We express our deepest gratitude to our project supervisor, **Ruchi Gupta**, Department of Information Technology, Ajay Kumar Garg Engineering College, Ghaziabad, for their invaluable guidance, constant encouragement, and constructive feedback throughout the course of this project. Their expertise and insights have been instrumental in shaping this work.

We sincerely thank **Dr. Rahul Sharma**, Head of the Department of Information Technology, for providing the necessary infrastructure, lab facilities, and an environment conducive to research and learning.

We extend our heartfelt thanks to the faculty members of the Department of Information Technology for their continuous support and for imparting the foundational knowledge that has made this project possible.

We are grateful to the management of **AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD** for maintaining an ecosystem that encourages innovation and applied research.

An AI assistant (Anthropic's Claude) was used in developing parts of the software and in drafting and editing this report and the related research paper; we have reviewed the content and take responsibility for it.

Finally, we acknowledge our families and friends for their unwavering moral support and patience during the entire duration of this project.

Student Names:

Madhav Goel
Akash Varshney
Abhisekh Chaudhary
Shivam Goel

Date: October 27, 2026

Abstract

Fixed-time traffic signals give every approach of a junction the same green time regardless of demand, which wastes green on lightly used approaches and lengthens queues on busy ones. This project designs, implements and evaluates a density-aware signal controller for an isolated four-arm junction. Each approach is scored from vehicle class and queue state, with a queued vehicle counting three times as much as a moving one; the green is extended between 8 s and 45 s while the served direction dominates, yellow and all-red intervals are never shortened, and an emergency-vehicle preemption layer keeps full change intervals.

The system is a complete simulation-first software stack: a Python microscopic simulator, a decision engine, a Haar-cascade detection and ByteTrack-style tracking pipeline, a FastAPI backend with a SQLite store, a React dashboard and an optional Raspberry Pi/ESP32 signal driver. It was evaluated on paired, seeded traffic against the fixed plan and four re-implemented strategies (Webster-optimized, actuated, max-pressure and three-level density control) in 3,524 runs.

Below capacity the controller reduced mean stopped delay by 35.3–36.1% relative to a 20 s fixed plan, but the Webster, actuated and max-pressure strategies did as well. Near capacity, density-only switching lost 18.9% of capacity; a discharge-hold rule restored it (+9.8% relative to fixed timing). The queued/moving distinction, not the class weights, drives the score's benefit. The findings hold with kinematics calibrated to field saturation headways once the hold zone is re-tuned, but with the present Haar detector (4.1% precision, 8.2% recall) every controller would be worse than fixed timing near capacity. A per-step audit found no conflicting greens or shortened change intervals in any run.

Keywords: adaptive traffic signal control, vehicle density, actuated control,

max-pressure control, emergency vehicle preemption, oversaturation, microscopic simulation

Contents

Declaration i

Certificate ii

Acknowledgements iii

Abstract iv

1 Introduction 1

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Research Objectives 2

1.4 Novelty and Originality 2

1.5 Scope of the Project 3

1.6 Report Organisation 3

2 Literature Review 4

2.1 Historical Overview 4

2.2 Recent and State-of-the-Art Work 4

2.3 Comparative Analysis of Existing Approaches 5

2.4 Identification of Research Gap 6

3 Research Methodology 7

3.1 Research Approach 7

3.2 Proposed System / Algorithm / Framework 7

3.2.1 Architecture and Workflow 7

3.2.2 Junction and Traffic Model 9

3.2.3 Density Estimation 10

3.2.4 Adaptive Green Extension 10

3.2.5 Discharge Hold 11

3.2.6 Comparison Controllers 11
3.2.7 Emergency Vehicle Preemption 12
3.2.8 Vision Pipeline 13
3.3 Tools, Technologies, and Platforms 13
3.4 Data Collection and Analysis 14
3.5 Evaluation Metrics 14

4 Implementation **16**
4.1 System Architecture Overview 16
4.2 Module-wise Description 16
4.2.1 Module 1: Simulation Engine 16
4.2.2 Module 2: Decision Engine 17
4.2.3 Module 3: Vision Pipeline 17
4.2.4 Module 4: Backend and Analytics 17
4.2.5 Module 5: Dashboard and Hardware Bridge 18
4.3 Database / Data Model 18
4.4 Key Algorithms 19
4.5 User Interface 20
4.6 Development and Deployment 20

5 Results and Discussion **21**
5.1 Experimental Setup 21
5.2 Results 21
5.2.1 Signal Safety Audit 21
5.2.2 Experiment 1: Comparison Below Capacity 21
5.2.3 Experiment 2: Capacity and Operation Above Capacity 22
5.2.4 Experiment 3: Sixty-Minute Peak Period 24
5.2.5 Experiment 4: Emergency Preemption 25
5.2.6 Experiment 5: Ablation 26
5.2.7 Experiment 6: Vision Pipeline 26
5.2.8 Experiment 7: Validation of the Simulated Kinematics 27

5.2.9 Experiment 8: Imperfect Sensing 28
5.2.10 Experiment 9: Calibration of the Density Weights . . 29
5.3 Discussion 30
5.4 Comparison with Existing Work 31
5.5 Limitations 31

6 Societal Impact and SDG Alignment 33
6.1 Primary SDG Relevance 33
6.2 Secondary SDG Linkages 33
6.3 Stakeholder Impact Analysis 34
6.4 Ethical Considerations 34

7 Universal Human Values and Harmony with Nature 36
7.1 Impact on the Material Order 36
7.2 Impact on the Plant / Bio Order 36
7.3 Impact on the Animal Order 36
7.4 Impact on the Human Order 37
7.5 Overall Mutual Fulfilment 37

8 Conclusion and Future Scope 38
8.1 Summary 38
8.2 Conclusions 38
8.3 Original Contributions 39
8.4 Limitations 40
8.5 Future Scope 40

References 42

A SDG Compliance Certificate 45

B Project Snapshots and Description 46
B.1 Module 1: Density Score (`decision_engine/density.py`) 46
B.2 Module 2: Adaptive Controller with Discharge Hold (`decision_engine/`

B.3 Module 3: Database Models (`database/models.py`) . . 47

List of Figures

3.1 Software architecture. Solid boxes form the control loop; dashed boxes form the vision pipeline, whose output is displayed and evaluated but not used for control in this study. . 8
3.2 Method workflow. (a) The per-step control pipeline. (b) The green-end decision; the test marked “discharge hold” is the extension added to the original controller. 8
3.3 Emergency preemption states. Every path from one direction’s green to the other’s passes through a full yellow and all-red interval. 12

5.1 Delay (top, log scale) and throughput (bottom) against demand, preemption off. The dotted line marks the fixed-time capacity. 24
5.2 Queue including vehicles waiting to enter during a 60-minute peak period (mean of ten seeds, 2-minute moving average, preemption off). Shading: demand ramping (light) and at 1.2 times the fixed-time capacity (dark). 25

List of Tables

2.1 Comparison of Existing Approaches 5

3.1 Vehicle classes and parameters used in the simulation 9
3.2 Tools and Technologies Used 13

4.1 Backend interfaces 18
4.2 Database tables 19

5.1 Mean stopped delay of the seven controllers below capacity (preemption enabled) 22
5.2 Capacity of each controller (veh/h) 22
5.3 Delay including vehicles still waiting to enter (s), preemption off 23
5.4 Sixty-minute peak period: queue, recovery, spillback, throughput and total delay 25
5.5 Ablation of the adaptive controller 26
5.6 Detector accuracy and ablation (synthetic frames) 27
5.7 Queue discharge against field reference values 28
5.8 Delay under imperfect sensing (s): medium demand / 0.9 times capacity 29
5.9 Comparison with the strategies of existing work (re-implemented in the same simulator) 31

6.1 Stakeholder Impact Matrix 34

Chapter 1

Introduction

1.1 Background and Motivation

Many junctions, especially in smaller cities, still run fixed-time signal plans in which each approach receives a preset green interval regardless of how many vehicles are waiting. Such plans are dimensioned for average flows; when demand fluctuates or is uneven, green time is wasted on nearly empty approaches while queues build on busy ones. Traffic in Indian cities adds variety: motorcycles, auto-rickshaws, buses and trucks share the road with cars, and emergency vehicles must cross a junction without delay.

Traffic-responsive systems such as SCATS and SCOOT adjust timings from detector data but rely on dedicated detector infrastructure [1, 2]. Low-cost alternatives estimate density locally with infrared sensors, image-based density classes or camera detectors [3, 4, 5, 6, 7, 8]. These systems are usually demonstrated on a prototype or a few scenarios and report an average improvement over a fixed plan. Four questions are rarely answered: whether variable greens keep safe change intervals; how emergency priority interacts with density-based timing; how the controller behaves as demand approaches and exceeds capacity; and how much of the gain is due to the density logic rather than to replacing an untuned plan with any demand-responsive one.

1.2 Problem Statement

The lack of a rigorous, reproducible evaluation of low-cost density-based signal control, against strong baselines, near and above capacity, and with verified signal safety, leaves it unclear when such a controller is actually better than a well-tuned fixed or actuated plan. This project addresses it by building a complete simulation-first traffic management system and evaluating its

density-aware controller on identical seeded traffic against re-implementations of the strategies used by existing approaches.

1.3 Research Objectives

The key objectives of this project are:

1. To design and develop a density-aware adaptive signal controller, an emergency-vehicle preemption layer that keeps full change intervals, and a vision pipeline, with a live dashboard and a hardware bridge.
2. To analyse and evaluate the controller against a fixed plan and four re-implemented strategies (Webster-optimized, actuated, max-pressure and three-level density control) below, at and above capacity.
3. To validate the system: signal safety at every step, simulated queue discharge against Highway Capacity Manual values, the vision pipeline against exact ground truth, and the effect of imperfect sensing.
4. To demonstrate the societal relevance of the work in line with the UN Sustainable Development Goals.

1.4 Novelty and Originality

Rather than a new detector, this project studies the *timing logic* of a density-based controller under paired, seeded conditions against re-implementations of a Webster-optimized plan [9], vehicle-actuated control [10], max-pressure control [11] and the three-level density control of [3]. It exposes a failure mode of density-only switching near capacity and corrects it with a discharge-hold rule taken from actuated control. The role of AI is stated plainly: the controller is a transparent rule, not a learned policy; machine learning enters through the pre-trained Haar-cascade detector, whose accuracy is measured, and through a data-driven calibration of the score on held-out seeds. The contributions are listed in Section 8.3.

1.5 Scope of the Project

Included: a microscopic simulator of a four-arm junction with seven vehicle classes; fixed-time, adaptive and four comparison controllers; emergency preemption with a per-step safety audit; a Haar-cascade detector and ByteTrack-style tracker evaluated on synthetic frames; a FastAPI backend with SQLite persistence; a React dashboard; a Raspberry Pi/ESP32 driver; Docker packaging and CI; and the experiments of Chapter 5.

Excluded: turning movements, lane changes, pedestrians and multi-junction coordination; real camera footage; closing the control loop on detector output; reinforcement learning; and tests on a physical junction or board (Section 8.5).

1.6 Report Organisation

Chapter 2 presents the literature review. Chapter 3 describes the research methodology. Chapter 4 covers the implementation details. Chapter 5 presents results and discussion. Chapter 6 discusses societal impact and SDG alignment. Chapter 7 addresses Universal Human Values. Chapter 8 concludes the report with future directions.

Chapter 2

Literature Review

2.1 Historical Overview

Fixed-time plans are commonly derived from Webster's delay-minimizing cycle formula [9]. SCATS adjusts splits, cycle lengths and offsets from detector data [1], and SCOOT optimizes the plan on line from loop-detector flows [2]. Vehicle-actuated control extends a green while stop-line detectors keep registering vehicles and ends it when a gap appears ("gap-out"), within minimum and maximum green limits [10]. Max-pressure control chooses phases from queue differences and has provable throughput guarantees [11]. On the simulation side, the Gipps model [12] and the Krauß model with a driver reaction time [13] underlie many simulators, SUMO among them [14], and the Highway Capacity Manual [15] supplies field saturation-flow values. For sensing, boosted cascades of Haar-like features [16] remain attractive for embedded use because inference is cheap, and multi-object trackers combine a Kalman filter [17] with Hungarian assignment [18], as in ByteTrack [19].

2.2 Recent and State-of-the-Art Work

Reinforcement-learning controllers are widely studied but must be trained in simulation and are harder to bound [20]. Nsofwa and Tembo classify image-based density into low, medium and high levels and adjust signal duration, showing in simulation that this outperforms fixed timing [3]. Kamble et al. apply YOLO-based recognition [21] to IP-camera feeds to prioritize the busiest lane [5]; Pardeshi et al. use passive infrared sensors on an ESP32 [4]; Patil et al. and Thomas et al. adapt timing to measured density [7, 8]; and Hu et al. estimate density from surveillance cameras [6]. Related systems add incident alerts [22], AI-based light design [23] or traffic prediction [24], and Nellore

and Hancke give emergency vehicles priority from visual sensing [25]. These works mainly report an improvement over fixed timing on a prototype or a few scenarios. This project adds strong baselines on identical traffic, demand up to 1.5 times capacity and a per-step safety check.

2.3 Comparative Analysis of Existing Approaches

Table 2.1: Comparison of Existing Approaches

Author(s)	**Year**	**Approach**	**Setting**	**Limitation (for this study)**
Webster [9]	1958	Delay-optimal fixed cycle and splits	Fixed plan from mean demand	Cannot react to fluctuating or uneven demand
Sims and Dobinson [1]; Hunt et al. [2]	1980, 1982	SCATS and SCOOT: detector-based adaptive plans	Networks with loop detectors	Needs dedicated detector infrastructure
Koonce et al. [10]	2008	Vehicle-actuated control with gap-out	Stop-line detectors	Needs detectors; no density score
Varaiya [11]	2013	Max-pressure phase selection	Queue differences	Moving vehicles also leave the pressure count as a queue discharges
Nsofwa and Tembo [3]	2025	Three-level image-based density classes set the green	Simulation	Compared with fixed timing; no capacity or safety-interval study reported
Pardeshi et al. [4]	2026	Infrared sensing and density-based prioritization on ESP32	Prototype platform	Gain reported against fixed timing
Kamble et al. [5]	2025	YOLO on IP-camera feeds; busiest lane first	Camera feeds	Detector is demanding on low-power hardware
Hu et al. [6]	2023	Density estimation from surveillance cameras	Existing camera network	Estimates density; timing logic not the focus
Patil et al. [7]; Thomas et al. [8]	2024, 2025	Timing adapted to measured density	Prototype or few scenarios	Gain reported against fixed timing

2.4 Identification of Research Gap

Based on the review above, the following gaps remain in the existing literature:

1. Gap 1: Low-cost density-based controllers are usually compared only with fixed timing, so it is unknown how much of the gain comes from the density logic.
2. Gap 2: Behaviour near and above capacity, and the safety of change intervals under variable greens and emergency preemption, are rarely examined.
3. Gap 3: The gap between idealized sensing and a real, inexpensive detector is seldom quantified, nor are the density weights validated.

The proposed project addresses these gaps by comparing against re-implemented Webster, actuated, max-pressure and density-level strategies on identical traffic, by sweeping demand from 0.6 to 1.5 times capacity with a per-step safety audit, and by measuring the effect of imperfect sensing and of calibrating the weights on held-out seeds.

Chapter 3

Research Methodology

3.1 Research Approach

The work is simulation-based and experimental. Experiments on a real junction are costly, unsafe and not repeatable, whereas a simulator lets every controller see exactly the same arrivals: a seed fixes the random arrival sequence, so runs of different controllers form matched pairs, reported as means with 95% confidence intervals and compared with paired t-tests. Control uses exact simulated vehicle states; the vision pipeline is evaluated alongside against exact ground truth, and a separate experiment measures what imperfect sensing would cost.

3.2 Proposed System / Algorithm / Framework

3.2.1 Architecture and Workflow

The system consists of Python packages with one-directional dependencies: the simulation core imports no decision or web code, and every signal controller implements one interface (an update per time step and a phase name), so controllers are interchangeable. Figures 3.1 and 3.2 show the architecture and the method: in every step the controller senses vehicle positions, speeds and classes, scores each approach, and decides whether the green may end; the preemption layer then protects every transition with full yellow and all-red intervals.

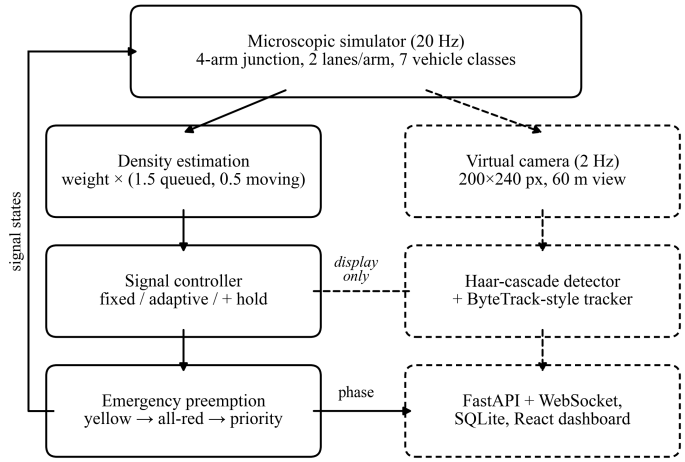


Figure 3.1: Software architecture. Solid boxes form the control loop; dashed boxes form the vision pipeline, whose output is displayed and evaluated but not used for control in this study.

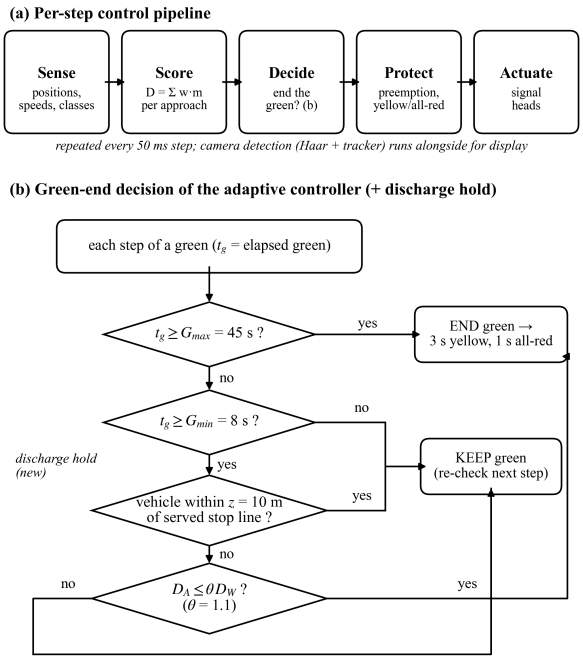


Figure 3.2: Method workflow. (a) The per-step control pipeline. (b) The green-end decision; the test marked “discharge hold” is the extension added to the original controller.

3.2.2 Junction and Traffic Model

The junction has four approaches, each with two lanes on a 330 m link; the stop line is at 200 m and the far edge of the junction box at 218 m. Vehicles travel straight through; turns, lane changes and pedestrians are not modelled. Vehicles arrive on each approach at a mean rate λ with deterministic headways; every 15 s the rate of each approach is redrawn as $\lambda \cdot U(0.4, 1.6)$, with a floor of 0.02 veh/s. A vehicle enters a lane at rest and is dropped if the rear of the previous vehicle is within 8 m. For the above-capacity experiments, blocked arrivals instead wait in an entry queue (so unserved demand is measured) and enter at the highest speed from which they can still stop behind their leader. Table 3.1 lists the seven vehicle classes.

Table 3.1: Vehicle classes and parameters used in the simulation

Class	**Share (%)**	**Length (m)**	v_{max} **(m/s)**	a **(m/s^2)**	b **(m/s^2)**	**Weight** w
Car	55	4.5	14	2.5	4.5	1.0
Motorcycle	25	2.0	15	3.5	5.0	0.5
Auto-rickshaw	10	3.2	11	2.0	4.0	0.8
Bus	5	10.0	10	1.2	3.0	3.0
Truck	4	9.0	9	1.0	2.8	2.5
Ambulance	0.5	5.5	16	3.0	5.0	6.0
Fire truck	0.5	10.5	12	1.5	3.5	6.0

a: maximum acceleration; b: maximum deceleration; w: density weight used in Eq. (3.3).

Longitudinal motion follows a Gipps-type safe-speed rule [12]. A follower's speed is limited to

$$u_{safe} = \sqrt{2b\left(g + \frac{u_l^2}{2b_l} - s_0\right)} \tag{3.1}$$

where g is the gap to the leader's rear, u_l and b_l are the leader's speed and maximum deceleration, b is the follower's maximum deceleration and $s_0 = 2$ m is the standstill gap. Adding a driver reaction time τ, as in Krauß's

model [13], gives

$$u_{safe}(\tau) = -b\tau + \sqrt{(b\tau)^2 + b\,\frac{u_l^2}{b_l} + 2b\,(g - s_0)} \tag{3.2}$$

which reduces to Eq. (3.1) for $\tau = 0$. The main results use $\tau = 0$; Section 5.2.8 shows that this makes queues discharge too fast and repeats the comparisons with $\tau = 1.6$ s. On yellow or red, a vehicle upstream of the stop line brakes once its braking distance reaches the distance to the line.

3.2.3 Density Estimation

For approach r with vehicle set V_r, the density score is

$$D_r = \sum_{v \in V_r} w_{c(v)}\, m(v) \tag{3.3}$$

where $w_{c(v)}$ is the class weight (Table 3.1) and $m(v) = 1.5$ if vehicle v is queued (upstream of the stop line and slower than 0.5 m/s) and 0.5 otherwise. The weights reflect the road space and discharge time of each class, and emergency vehicles receive a high weight; a queued vehicle counts three times as much as a moving one because it needs green now. The weights are heuristic; Sections 5.2.6 and 5.2.10 test and calibrate them. For display, an approach is labelled low ($D_r \le 3$), medium ($3 < D_r \le 6$) or high ($D_r > 6$).

3.2.4 Adaptive Green Extension

The controller uses a two-phase plan: north–south green, yellow ($Y = 3$ s), all-red ($A = 1$ s), then east–west green, yellow and all-red. Only the green adapts. With t_g the elapsed green and D_A, D_W the summed scores of the active and waiting direction, the green ends when

$$t_g \ge G_{max} \quad \text{or} \quad \big(t_g \ge G_{min} \text{ and } D_A \le \theta D_W\big) \tag{3.4}$$

with $G_{min} = 8$ s, $G_{max} = 45$ s and $\theta = 1.1$. Because yellow and all-red are fixed and green is capped, the red time of any approach in normal operation

is bounded by

$$R_{max} = A + G_{max} + Y + A = 50 \text{ s} \tag{3.5}$$

so no approach can be starved. The fixed-time baseline uses the same sequence with a 20 s green (48 s cycle).

3.2.5 Discharge Hold

When both directions are saturated, vehicles moving off on green score 0.5 each while the queue growing on red scores 1.5 each, so $D_A \leq \theta D_W$ holds soon after G_{min} and greens stay short; each phase change costs $Y + A = 4$ s with no vehicle served, so capacity is lost exactly when it is scarcest (Section 5.2.3). The discharge hold adds the gap-out test of actuated control [10]: past the minimum, a green may only end once no served vehicle is within z of its stop line,

$$t_g \geq G_{max} \quad \text{or} \quad \big(t_g \geq G_{min} \text{ and } n_z = 0 \text{ and } D_A \leq \theta D_W\big) \tag{3.6}$$

where n_z counts served vehicles within $z = 10$ m of the stop line (chosen from 10–30 m on a separate tuning seed). A saturated queue keeps a vehicle at the line and holds its green up to G_{max}, while a cleared queue still hands over early; bound (3.5) still holds.

3.2.6 Comparison Controllers

Four strategies were re-implemented in the same simulator with the same phases, change intervals and 8–45 s green limits, so every one obeys bound (3.5) and only the switching decision differs. Like the proposed controller they read exact vehicle states; they are not the original systems.

- **Webster fixed:** green from Webster's formula [9], $C_0 = (1.5L{+}5)/(1{-}Y_f)$ with lost time $L = 8$ s, Y_f the sum of the critical flow ratios, saturation flow $s = 6000$ veh/h per approach (measured in the simulator) and each phase receiving $(C_0 - L)/2$, clamped to 8–45 s. It is given the true mean demand of each run, which favours it.

- **Actuated:** past the minimum, the green ends when the served direction gaps out (no vehicle within 10 m of its stop line) and a vehicle waits on the other direction [10].
- **Max-pressure:** switch once the waiting phase's upstream vehicle count exceeds the served one's [11]; this also represents busiest-lane-first control [5].
- **Density levels:** following [3], the green is set at its start to 10, 20 or 30 s for low, medium or high density.

3.2.7 Emergency Vehicle Preemption

Preemption wraps any phase-based controller (Figure 3.3). When an ambulance or fire truck is upstream of the junction exit: a conflicting green first shows a full 3 s yellow and then 1 s all-red (a green shown for less than 5 s is never cut); the emergency vehicle's direction then has green until no emergency vehicle remains before the exit, or for at most 20 s; and the release again passes through full yellow and all-red before the inner controller restarts. An emergency vehicle thus waits at most $5 + 3 + 1 = 9$ s. Detection is assumed perfect, e.g. from vehicle-to-infrastructure messages.

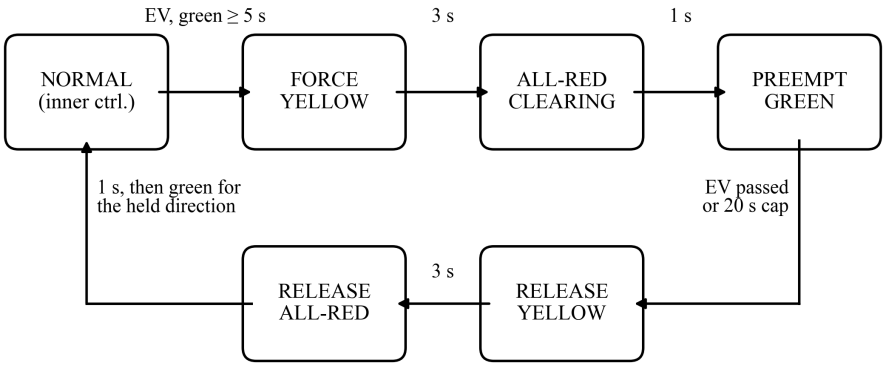


Figure 3.3: Emergency preemption states. Every path from one direction's green to the other's passes through a full yellow and all-red interval.

3.2.8 Vision Pipeline

A virtual camera per approach renders a 200×240 pixel frame of the 60 m before the stop line (4 px/m, 2 Hz). The detector equalizes the histogram, upscales three times and applies OpenCV's pre-trained vehicle Haar cascade [16] (scale factor 1.05, minimum neighbours 2, minimum size 20 px); it localizes vehicles but does not classify them. The tracker keeps a constant-velocity Kalman filter [17] per box centre and matches predicted boxes to detections with the Hungarian algorithm [18] on an IoU cost (minimum 0.15); as in ByteTrack [19], detections with confidence ≥ 0.15 are matched first and the rest are then matched to unmatched tracks. A track is confirmed after two matches and deleted after three frames without one.

3.3 Tools, Technologies, and Platforms

Table 3.2: Tools and Technologies Used

Category	**Tool / Technology**	**Purpose**
Language	Python 3.12; JavaScript	Simulation, decision engine, backend; dashboard
Numerical / Vision	NumPy 2.1, SciPy 1.14, OpenCV 4.10	Statistics, Hungarian assignment; Haar cascade
Backend	FastAPI 0.115, Pydantic 2.9, WebSockets	REST API and live streaming
Frontend	React 18, Vite 5, Tailwind 3, Recharts 2	Live dashboard and analytics charts
Database	SQLite via SQLAlchemy 2.0 (PostgreSQL-ready)	Snapshots, events, report export
Testing / Packaging	pytest 8, GitHub Actions, Docker Compose	119 automated tests; CI; deployment
Hardware (optional)	Raspberry Pi (GPIO) / ESP32	Mirror the signal on LEDs or relays

3.4 Data Collection and Analysis

No real images or field data are used. Traffic is generated by the simulator with seeded arrivals, and camera frames and their ground-truth boxes are both rendered from the same simulator state, so annotation is automatic and exact. Nothing is trained: the cascade is pre-trained, the controller is rule-based, and detector and tracker parameters were fixed during development in unseeded runs, so there is no training split. Seed 1000 was used for tuning (the hold zone), seeds 101–105 for capacity, and seeds 1–10 for all reported results (1–4 for vision); in total 3,524 runs (1,797 simulated hours). Three demand levels below capacity were simulated ($\lambda = 0.08, 0.12, 0.18$ veh/s per approach, i.e. 1152, 1728 and 2592 veh/h), each for ten seeds of 1800 s with a 50 ms step and a 60 s warm-up; delay statistics use vehicles entering between 60 s and 1650 s. For the above-capacity experiments, capacity was first measured by flooding every approach (1.5 arrivals/s; five seeds), and demand was then set to 0.6, 0.8, 0.9, 1.0, 1.2 and 1.5 times the measured fixed-time capacity.

3.5 Evaluation Metrics

The following metrics are used to evaluate the proposed system:

- **Stopped delay:** time a vehicle spends upstream of the stop line below 0.5 m/s (including a start-up component of 0.15–0.55 s that is the same for all controllers); and **delay including waiting vehicles**, a lower bound above capacity.
- **Throughput and capacity:** vehicles served per hour after the warm-up; capacity is the throughput with every approach flooded.
- **Queue growth and spillback:** growth rate of waiting vehicles (veh/min), the share of time the queue reaches the start of the link, and the time to return to the pre-peak queue.

- **Signal safety:** conflicting greens, missing yellows, and yellows or all-reds shorter than 3 s or 1 s, checked at every step of every run.
- **Detection and tracking:** precision and recall (correct if the centre lies inside a ground-truth box enlarged by 4 px), tracking coverage and tracked-speed error.
- **Kinematic validity:** saturation headway, flow and start-up lost time against the Highway Capacity Manual.
- **Statistics:** 95% confidence intervals and paired t-tests over seeds.

Chapter 4

Implementation

4.1 System Architecture Overview

The implementation is a layered, simulation-first system (about 3,600 lines of Python, excluding tests and experiment scripts, plus a React front end), built milestone by milestone. A headless engine advances the junction in 50 ms steps (20 Hz): in each step the active controller sets the light of every approach and then all vehicles move. The decision engine scores each approach and decides when a green ends; the preemption wrapper overrides it when an emergency vehicle approaches. A vision pipeline renders a virtual camera frame per approach at 2 Hz for the detector and tracker; its output is shown on the dashboard and compared with ground truth, but control uses the simulator's vehicle states (ideal detection). A FastAPI backend runs the simulation, streams the state over WebSockets at 10 Hz and stores snapshots; a React dashboard draws the junction; and an optional driver mirrors the signal on real lights (Figure 3.1).

4.2 Module-wise Description

4.2.1 Module 1: Simulation Engine

Package: `simulation/`. It holds the vehicle, lane, road, traffic-light and intersection entities, the engine loop and the controller interface. Vehicles of seven classes (Table 3.1) follow the safe-speed rule (3.1), with an optional reaction time (3.2). The `SignalControllerBase` strategy interface is the one seam that lets the fixed-timer, adaptive, comparison and preemption controllers plug in without changing the engine, backend or front end. Optional behaviours used by the experiments, all off by default, are the entry

queue, entry at the highest safe speed, the reaction time, a time-varying demand profile and a sensor model. A `SignalAudit` checks every step for conflicting greens and shortened change intervals.

4.2.2 Module 2: Decision Engine

Package: `decision_engine/`. `density.py` implements the score of Eq. (3.3); `adaptive_controller.py` the green extension of Eq. (3.4) and, with `hold_while_discharging`, the discharge hold of Eq. (3.6); `baselines.py` and `webster.py` the four comparison controllers, which reuse the phase sequence and change intervals and replace only the decision of whether a green may end; `emergency_preemption.py` the preemption wrapper of Figure 3.3; and `sensing.py` a wrapper that lets a controller see the junction through a sensor model (limited view, refresh rate, missed detections, no classes, phantom detections).

4.2.3 Module 3: Vision Pipeline

Package: `vision/`. `camera.py` renders the virtual camera frames, `detector.py` is the Haar-cascade detector, `tracker.py` the ByteTrack-style tracker, and `pipeline.py` compares detections and tracks with ground truth. In two single-process runs of 400 frames on a 12th Gen Intel Core i5-12450H laptop CPU, detection took 103 and 171 ms per frame on average (it varies with the CPU's power state), whereas a control decision took 0.002–0.004 ms.

4.2.4 Module 4: Backend and Analytics

Packages: `backend/app/`, `analytics/`. A FastAPI application runs the simulation, vision and analytics tasks and exposes REST endpoints and WebSocket channels (Table 4.1). The analytics collector samples queue, delay, throughput and congestion level at 1 Hz, raises a high-traffic alert when an approach stays at high density for 3 s, and exports reports as CSV or JSON. A comparison runner executes a second simulation with the same seed under

another controller, so the dashboard can show fixed and adaptive control side by side.

Table 4.1: Backend interfaces

Endpoint	**Purpose**
GET /api/health	Liveness check (Docker health check)
GET /api/intersection/state	Current junction state
GET /api/vision/latest	Latest detections and tracks
GET /api/analytics/history/{road}	Time series for one approach
GET /api/analytics/summary	Aggregate queue, delay, throughput, congestion
GET /api/reports/export	Report export as CSV or JSON
GET /api/comparison/latest	Fixed-versus-adaptive comparison
GET /api/explain	Explanation of each displayed quantity
GET /api/signals/status	Signal state polled by the hardware driver
WS /ws/simulation, /ws/simulation/shadow, /ws/vision, /ws/analytics, /ws/comparison	Live junction state, second simulation, detections, analytics and comparison

4.2.5 Module 5: Dashboard and Hardware Bridge

Directories: `frontend/`, `hardware/`. The React dashboard (Section 4.5) consumes the WebSocket channels. The hardware bridge is a Python driver for a Raspberry Pi and an Arduino/C++ sketch for an ESP32 that poll `GET /api/signals/status` at 5 Hz and drive 12 LEDs (red, yellow and green for four approaches, each through a 220 Ω resistor); larger lamps need an opto-isolated relay. Since the driver is only a client, nothing in the simulation, decision engine or backend changes. The bridge has not yet been built and tested on a physical board.

4.3 Database / Data Model

Persistence uses SQLAlchemy 2.0 with SQLite by default (PostgreSQL-ready). Splitting a snapshot into two tables avoids repeating each road's columns in one wide table and matches the analytics collector's per-road queries (Table 4.2). A snapshot is stored every 10th one-hertz sample, i.e. every 10 s,

and signal events are recorded as they occur; in Docker the SQLite file lives in a named volume, so history survives a restart.

Table 4.2: Database tables

Table	**Columns**
simulation_runs	id, started_at, ended_at, controller_type
snapshots	id, run_id (FK), sim_time, captured_at
road_snapshots	id, snapshot_id (FK), road_name, queue_length, vehicle_count, density_score, light_state, has_emergency_vehicle, total_exited, total_wait_time_s
signal_events	id, run_id (FK), sim_time, road_name, event_type, phase_name, details (JSON), created_at

4.4 Key Algorithms

The core algorithms are the density score, the green-end decision with discharge hold, and the preemption state machine. The green-end decision evaluated at every step is, in pseudocode:

Listing 4.1: Green-end decision with discharge hold

```
def green_may_end(t_g, D_A, D_W, near_stop_line):
    if t_g >= G_MAX:                        # 45 s: hard cap
        return True
    if t_g < G_MIN:                         # 8 s: minimum green
        return False
    if HOLD and near_stop_line:             # a served vehicle within
        z = 10 m
        return False                        # queue still discharging
            : keep green
    return D_A <= THETA * D_W               # theta = 1.1
```

The preemption wrapper moves through *inner controller*, *entry yellow*, *all-red clearing*, *priority green*, *release yellow* and *release all-red*, so every path from one direction's green to the other's passes through a full yellow and all-red (Figure 3.3); the per-step audit checks this in every run. Appendix B lists the source of the main functions.

4.5 User Interface

The React dashboard draws the junction to scale on an HTML5 canvas from the streamed state. Its components are the intersection canvas; signal-status and per-road statistics cards with density score and congestion level; a camera-feed grid with detection and tracking overlays; a traffic-alert banner; an analytics dashboard with historical graphs and report export; a comparison view that runs fixed and adaptive control on the same seed; and a "how it works" page and explain panel.

4.6 Development and Deployment

- **Operating System:** Windows 11 for development; Ubuntu in CI runners and Docker containers.
- **Key Libraries:** Python 3.12; FastAPI 0.115.0, SQLAlchemy 2.0.35, NumPy 2.1.2, SciPy 1.14.1, OpenCV (headless) 4.10.0.84, pytest 8.3.3; React 18.3, Vite 5.4, Tailwind CSS 3.4, Recharts 2.12.
- **Deployment:** `docker compose up -build` starts the backend (port 8000, with a health check) and the dashboard behind nginx (port 8080). A GitHub Actions workflow runs the 119-test suite, the front-end build and a Docker build with smoke test on every push.

Chapter 5

Results and Discussion

5.1 Experimental Setup

All experiments ran on the simulator of Chapter 3 on an Intel Core i5-12450H laptop, using the scripts in the `scripts/` folder, from which every table can be regenerated (setup and seeds as in Section 3; 3,524 runs, 1,797 simulated hours). Delay is the stopped delay, with 95% confidence intervals and paired t-tests over ten seeds.

5.2 Results

5.2.1 Signal Safety Audit

Every step of every run was checked for conflicting greens, missing yellows, and yellow or all-red shorter than 3 s or 1 s. Across all 3,524 runs, covering all seven controllers and 1,830 preemption events in the main runs alone, **none of these violations occurred**. The audit also exposed a flaw in an earlier preemption version that resumed without a release interval, which led to the release states of Figure 3.3.

5.2.2 Experiment 1: Comparison Below Capacity

In Table 5.1 the adaptive controller reduced mean stopped delay by 35.3–36.1% relative to the 20 s fixed plan, in all ten seeds at every level ($p < 0.001$), and the 95th-percentile delay fell by 37.1–41.2%. However, the Webster plan, actuated control and max-pressure control achieved the same or slightly lower delay (37.5–39.7%, 37.2–39.9% and 37.3–39.8% below the fixed plan; between -6.8% and -2.9% relative to adaptive). Below capacity, therefore, most of the gain comes from replacing an untuned 20 s plan with

short, demand-responsive greens (mean adaptive green 9.5 s at medium demand). Three-level density control was worse than the fixed plan at medium and high demand (+29.0% and +28.9%), as its 20–30 s greens outlast the queue. The discharge hold changed delay by between -1.2% and +2.5% relative to the adaptive controller, and throughput changed by at most 0.9%, since all levels are far below capacity.

Table 5.1: Mean stopped delay of the seven controllers below capacity (preemption enabled)

Controller	**Delay low (s)**	**Delay medium (s)**	**Delay high (s)**	**High vs fixed**	**95th pct. high (s)**	**Green (s)**
Fixed (20 s)	6.96 ± 0.17	7.20 ± 0.20	7.36 ± 0.22	—	23.6 ± 0.3	19.3
Webster fixed	4.29 ± 0.18	4.34 ± 0.15	4.60 ± 0.29	-37.5%	13.9 ± 1.5	8.7
Actuated	4.19 ± 0.21	4.36 ± 0.13	4.63 ± 0.17	-37.2%	13.5 ± 0.4	8.8
Max-pressure	4.22 ± 0.14	4.33 ± 0.17	4.62 ± 0.18	-37.3%	13.9 ± 0.6	8.9
Density levels	7.35 ± 0.37	9.29 ± 0.33	9.49 ± 0.15	+28.9%	32.0 ± 0.3	26.9
Adaptive	4.49 ± 0.10	4.60 ± 0.14	4.76 ± 0.12	-35.3%	14.8 ± 1.0	9.5
Adaptive + hold	4.44 ± 0.10	4.62 ± 0.16	4.89 ± 0.23	-33.7%	15.3 ± 1.5	9.7

Mean $\pm$ 95% confidence interval over ten paired seeds. All changes relative to the fixed plan are significant ($p < 0.001$) except density levels at low demand ($p = 0.06$). Green: mean green time at high demand.

5.2.3 Experiment 2: Capacity and Operation Above Capacity

The three levels of Table 5.1 are only 12–26% of the junction's measured fixed-time capacity (10,008 veh/h), so they cannot show congestion. Table 5.2 gives each controller's capacity; these runs and the steady sweep use preemption off, because emergency vehicles (1% of arrivals) would otherwise trigger a preemption every 15–30 s and dominate every controller's timing.

Table 5.2: Capacity of each controller (veh/h)

Controller	**Capacity, preemption off**	**vs fixed**	**Mean green (s)**	**Capacity, preemption on**
Fixed (20 s)	10008 ± 89	—	20.0	9701 ± 197
Webster fixed	11168 ± 32	+11.6%	45.0	9977 ± 224
Actuated	10971 ± 75	+9.6%	37.3	10018 ± 223
Max-pressure	8128 ± 136	-18.8%	8.1	8876 ± 296
Density levels	10637 ± 55	+6.3%	30.0	9976 ± 193
Adaptive	8112 ± 155	-18.9%	8.1	8861 ± 321
Adaptive + hold	10988 ± 41	+9.8%	37.6	10012 ± 222

Every approach flooded at 1.5 arrivals/s; five seeds; throughput after the 60 s warm-up.

The original adaptive rule lost 18.9% of capacity relative to fixed timing ($p < 0.001$): with both directions saturated its greens fell to the 8 s minimum (mean 8.1 s), as explained in Section 3.2.5. With the discharge hold the mean saturated green rose to 37.6 s and capacity to +9.8%. Max-pressure suffered the same loss as the original rule (-18.8%), since moving vehicles also leave its pressure count, whereas actuated control (+9.6%) and the Webster plan (+11.6%), which runs the maximum green above capacity, did not.

Demand was then swept from 0.6 to 1.5 times capacity with blocked arrivals kept in an entry queue (Table 5.3, Figure 5.1). The original rule beat fixed timing only at 0.6×: at capacity its delay was 148.7 s against 12.2 s, its queue grew by 28.9 vehicles per minute, and queues reached the link start during 78% of approach-time. With the hold, delay was below fixed timing at every level (-33.2%, -16.3%, -7.6%, -13.3%, -57.7%, -21.3% from 0.6× to 1.5×; not significant at 1.0×, $p = 0.14$) and throughput rose by +9.5% and +10.1% at 1.2× and 1.5×. Actuated control behaved like the hold (10.6 s at capacity and 52.8 s at 1.2×, against 10.6 s and 48.7 s), and max-pressure degraded like the original rule. With preemption on, the same pattern held with smaller margins (capacity -8.7% and +3.2%), because preemption also cuts the hold's long greens.

Table 5.3: Delay including vehicles still waiting to enter (s), preemption off

Controller	**0.6×**	**0.8×**	**0.9×**	**1.0×**	**1.2×**	**1.5×**
Fixed (20 s)	6.4	7.7	8.9	12.2	115.2	312.2
Webster fixed	4.8	8.2	11.5	17.1	50.6	240.4
Actuated	4.4	6.5	8.1	10.6	52.8	245.7
Max-pressure	4.6	9.7	25.6	132.0	292.9	438.8
Adaptive	4.6	10.1	41.5	148.7	298.2	440.1
Adaptive + hold	4.3	6.4	8.2	10.6	48.7	245.6

Demand as a multiple of the fixed-time capacity; mean over ten paired seeds. Vehicles still waiting at the end of a run count with the delay accrued so far, so values above capacity are lower bounds.

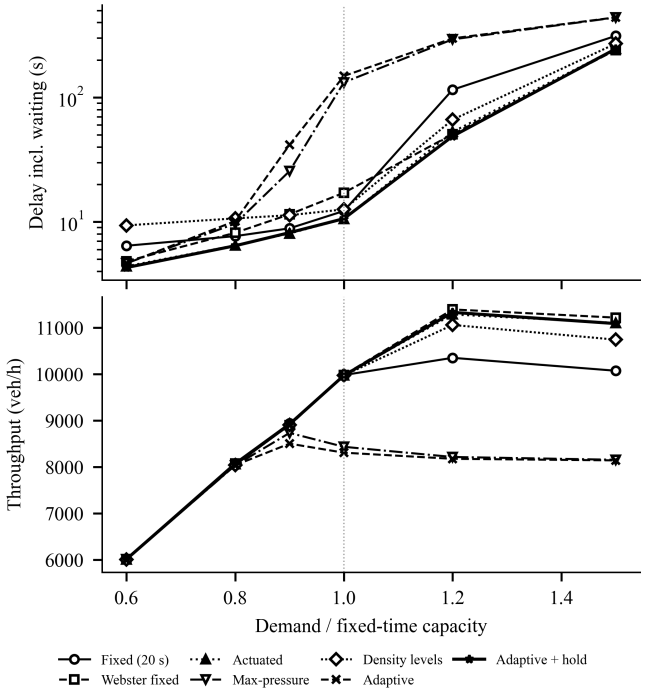


Figure 5.1: Delay (top, log scale) and throughput (bottom) against demand, preemption off. The dotted line marks the fixed-time capacity.

5.2.4 Experiment 3: Sixty-Minute Peak Period

In a 60-minute experiment, demand rose from 0.6 to 1.2 times capacity (ramping over 10–15 min), stayed there for 15 min and fell back to 0.6 by 35 min (Table 5.4, Figure 5.2; preemption off, ten paired seeds; the Webster plan timed for the average demand). The queue, including vehicles waiting to enter, peaked at 495 vehicles under fixed timing, 255 with the hold, 254 with actuated control and 1214 with the original rule. After the peak, the queue returned to its pre-peak level in 13.9 min under fixed timing and 7.3 min with the hold, whereas the original rule did not recover within the hour and max-pressure 28.7 min (9 of 10 runs did not recover). Entries were blocked for 68, 36 and 172 approach-minutes (fixed, hold, original); the hold served 11394 veh/h during the peak against 10574 for fixed timing and 8244 for the original rule; and total delay changed by -57.3% with the hold, -57.6% with actuated control and +356.1% with the original rule. Managing a congestion episode therefore depends above all on not shortening greens while queues

are saturated.

Table 5.4: Sixty-minute peak period: queue, recovery, spillback, throughput and total delay

Controller	Peak queue (veh)	Time to recover	Blocked (approach-min)	Peak throughput (veh/h)	Total delay vs fixed
Fixed (20 s)	495	13.9 min	68	10574	—
Webster fixed	444	13.1 min	64	10723	−10.5%
Actuated	254	7.4 min	36	11406	−57.6%
Max-pressure	1166	28.7 min (9 of 10 runs did not recover)	167	8374	+325.6%
Adaptive	1214	did not recover within the hour	172	8244	+356.1%
Adaptive + hold	255	7.3 min	36	11394	−57.3%

Demand ramps from 0.6 to 1.2 times capacity, holds for 15 min and falls back; ten paired seeds; preemption off.

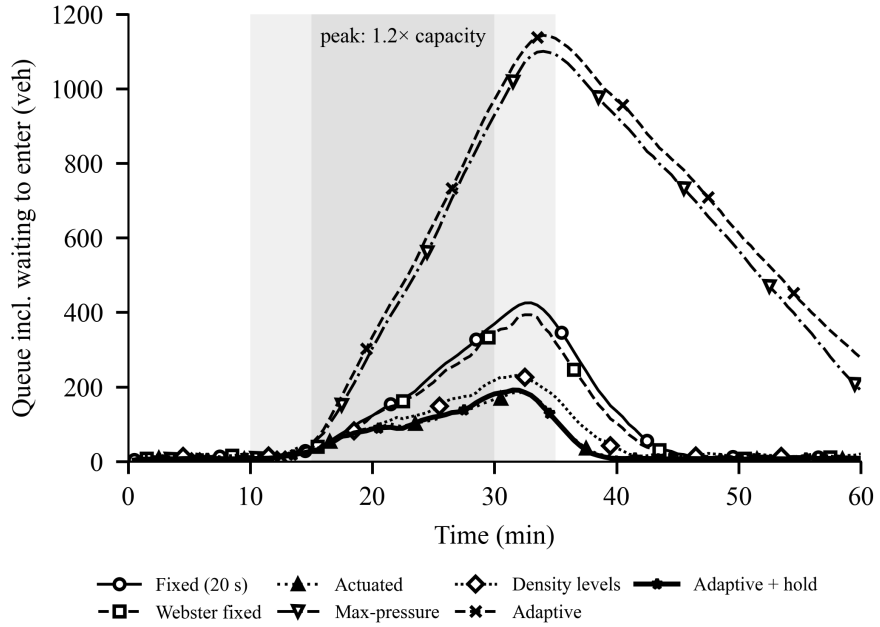


Figure 5.2: Queue including vehicles waiting to enter during a 60-minute peak period (mean of ten seeds, 2-minute moving average, preemption off). Shading: demand ramping (light) and at 1.2 times the fixed-time capacity (dark).

5.2.5 Experiment 4: Emergency Preemption

Emergency vehicles (1% of arrivals) triggered 12.4–23.4 preemptions per hour. At medium demand, preemption cut their mean stopped delay from 8.87 s to 0.28 s under fixed timing and from 3.65 s to 0.35 s under adaptive control ($p < 0.001$), but other traffic paid +3.1% under fixed timing, +9.3% under adaptive control and +7.2% with the hold, because each preemption discards the density-responsive timing. Emergency vehicles are far more fre-

quent here than in normal traffic, so the real cost would be smaller.

5.2.6 Experiment 5: Ablation

Table 5.5 changes one part at a time on the same seeds. Counting vehicles without weights or multipliers raised delay by 22.7% and 26.6% at medium and high demand; removing only the queued/moving distinction raised it by 35.1% and 33.0%, whereas setting every class weight to 1 changed it by -3.9% and -3.2%. The benefit of the score thus comes from distinguishing queued from moving vehicles, not from the class weights. Raising θ to 1.5 lowered delay slightly (-4.9% and -4.1%), and the hold, with any zone length, left delay unchanged below capacity.

Table 5.5: Ablation of the adaptive controller

Variant	**Delay medium (s)**	**Change (p)**	**Delay high (s)**	**Change (p)**
Adaptive (reference)	4.60 ± 0.14	—	4.76 ± 0.12	—
Unweighted count	5.65 ± 0.16	+22.7% (< 0.001)	6.03 ± 0.19	+26.6% (< 0.001)
No class weights	4.42 ± 0.14	-3.9% (0.07)	4.61 ± 0.15	-3.2% (0.04)
No queued/moving distinction	6.22 ± 0.21	+35.1% (< 0.001)	6.34 ± 0.15	+33.0% (< 0.001)
theta = 1.0	4.66 ± 0.13	+1.3% (0.39)	4.75 ± 0.14	-0.3% (0.84)
theta = 1.5	4.38 ± 0.13	-4.9% (0.005)	4.57 ± 0.14	-4.1% (0.06)
+ discharge hold, 10 m zone	4.62 ± 0.16	+0.4% (0.82)	4.89 ± 0.23	+2.5% (0.17)
+ discharge hold, 20 m zone	4.66 ± 0.14	+1.3% (0.44)	4.87 ± 0.15	+2.2% (0.06)
+ discharge hold, 30 m zone	4.61 ± 0.12	+0.2% (0.92)	4.94 ± 0.19	+3.6% (0.09)
Preemption off	4.24 ± 0.09	-7.8% (< 0.001)	4.22 ± 0.06	-11.4% (< 0.001)
Fixed 20 s (for reference)	7.20 ± 0.20	+56.5% (< 0.001)	7.36 ± 0.22	+54.5% (< 0.001)

Preemption enabled unless stated; ten paired seeds per row; change relative to the adaptive reference at the same demand.

5.2.7 Experiment 6: Vision Pipeline

On seeds 1–4 (9,600 frames, 14,874 vehicle instances) the generic cascade reached 4.1% precision and 8.2% recall (Table 5.6). 98.6% of the false positives recur at the same spot, i.e. the cascade fires on background texture rather than on vehicles; recall varies by class (car 7.1%, motorcycle 11.8%, auto 7.6%, bus 7.2%, truck 7.7%, fire_truck 3.7%, ambulance 14.3%) and distance (0-20 m 5.4%, 20-40 m 16.9%, 40-60 m 17.4%) and is lowest near

the stop line. Without upscaling or without histogram equalization the cascade found essentially nothing, and a 2× upscale raised precision to 63.3% but cut recall to 3.7%. Only 0.6% of vehicle instances carried a confirmed track, and tracked speed had a mean absolute error of 0.53 m/s over 82 estimates against vehicles moving at only 0.56 m/s, since tracks are confirmed almost only on queued vehicles: a negative result, as the pipeline gives no usable kinematics for moving traffic. Dropping the low-confidence matching stage or the motion model changed little, because recall is the limit. This is why the evaluated controllers read vehicle states from the simulator; accuracy on real footage is unknown.

Table 5.6: Detector accuracy and ablation (synthetic frames)

Detector variant	**Precision (%)**	**Recall (%)**	**Detections per frame**
Default (3x upscale, equalized)	4.1	8.2	3.30
2x upscale	63.3	3.7	0.09
No upscaling	—	0.0	0.00
No histogram equalization	—	0.0	0.00

Pooled over seeds 1–4; a detection is correct when its centre lies inside a ground-truth box enlarged by 4 px.
—: too few detections for a meaningful precision.

5.2.8 Experiment 7: Validation of the Simulated Kinematics

Queue discharge was measured as at a real junction: headways of queued vehicles crossing the stop line in saturated fixed-time runs (five seeds), averaged from the fifth vehicle on. The Highway Capacity Manual base is 1900 veh/h/lane (a 1.89 s headway) with about 2 s start-up lost time [15]. The model of the main results discharges cars every 0.51 s (7,102 veh/h/lane; Table 5.7), because without a reaction time a follower may trail by only the 2 m standstill gap. With $\tau = 1.6$ s in Eq. (3.2) the headway is 1.92 s (1,877 veh/h/lane), within 1.2% of the reference; start-up lost time (-0.8 s) stays below field values.

Table 5.7: Queue discharge against field reference values

Model	Headway, cars (s)	Sat. flow, cars (veh/h/lane)	Sat. flow, all (veh/h/lane)	Start-up lost time (s)
Simulator, $\tau = 0$ (used for main results)	0.51	7,102	3,164	2.1
Simulator, $\tau = 1.6$ s (calibrated)	1.92	1,877	1,798	-0.8
HCM base values [24]	1.89	1,900	—	≈ 2.0

Saturated fixed-time runs, five seeds; saturation headway averaged from the fifth queued vehicle on.

All comparisons were repeated with $\tau = 1.6$ s (Webster re-timed for 3600 veh/h). Below capacity, delay fell by 33.7–35.4% for the adaptive controller and 31.9–39.2%, 35.0–39.5% and 34.7–38.9% for Webster, actuated and max-pressure. Relative to the calibrated fixed-time capacity (6,148 veh/h), the original rule reached -14.6% and Webster +6.3%, but the hold with its 10 m zone only -3.7% (mean saturated green 17.4 s): vehicles leaving a queue now spread to gaps of about 20 m, so a 10 m zone empties between them and the green gaps out too early. The zone is thus a kinematics-dependent parameter, as the passage time of a real actuated controller is set on site. Re-tuned on the tuning seed to 20.0 m, the hold reached +6.1% and actuated control +6.1%, against +6.3% for Webster.

5.2.9 Experiment 8: Imperfect Sensing

To measure the cost of the exact-state assumption, the vehicle-reading controllers ran behind a sensor model that reports only what a camera would: vehicles within 60 m, refreshed at 2 Hz, without classes, each detected with a given recall, optionally with phantom detections at fixed spots like the cascade's static false positives, and finally the measured pipeline of Experiment 6; preemption reads true flags. Table 5.8 gives delay at medium demand and 0.9 times capacity (medium / 0.9×). Below capacity the controllers tolerated missed detections well, because missing vehicles only shortens greens, which costs little when queues are short. Near capacity, sensing mattered: the discharge-hold controller's delay rose from 7.8 s with exact states to 8.5 s with a perfect camera view and 14.4 s at 50% recall, against 8.8 s for the fixed plan. Phantom detections were the most damaging error for the gap-out rule: a phantom inside the 10 m zone never moves, so the green never

gaps out and runs to the maximum (medium-demand delay 11.9 s, +64.8% relative to fixed timing). With the measured pipeline, every vehicle-reading controller was worse than the fixed plan near capacity (discharge hold 162.9 s against 8.8 s). At medium demand max-pressure and adaptive still beat the fixed plan, because phantoms appear on both directions and largely cancel in their comparisons, whereas actuated and adaptive + hold were held at long greens by phantoms in the stop-line zone (11.9 s against 7.2 s). Closing the loop with the present detector would harm traffic; a detector must at least suppress static false positives near the stop line, and its recall matters most near capacity.

Table 5.8: Delay under imperfect sensing (s): medium demand / 0.9 times capacity

Sensing	**Actuated**	**Max-pressure**	**Adaptive**	**Adaptive + hold**
Exact states	4.4 / 8.1	4.3 / 27.2	4.6 / 35.6	4.6 / 7.8
Camera	4.4 / 8.5	4.3 / 30.1	4.3 / 45.0	4.4 / 8.5
recall 90%	4.4 / 8.4	4.4 / 32.3	4.3 / 45.4	4.3 / 8.3
recall 70%	4.3 / 9.0	4.3 / 36.4	4.3 / 45.4	4.4 / 9.4
recall 50%	4.4 / 11.3	4.3 / 33.0	4.3 / 45.4	4.3 / 14.4
recall 30%	4.4 / 20.6	4.4 / 35.7	4.4 / 48.6	4.4 / 23.2
2 phantoms	11.9 / 15.1	4.3 / 30.1	4.3 / 45.0	11.9 / 15.1
Haar (recall 8%, 3 phantoms)	11.9 / 163.3	4.7 / 33.2	4.3 / 45.2	11.9 / 162.9

Ten paired seeds. Fixed plan (needs no sensing): 7.2 ± 0.2 / 8.8 ± 0.2 s. The $0.9\times$ column excludes preemption and counts time waiting to enter the road.

5.2.10 Experiment 9: Calibration of the Density Weights

The hand-set weights were calibrated by random search over the class weights (0.25–4; emergency 1–10), the queued/moving ratio (1–12) and θ (1.0–1.6) of the hold controller, evaluating 14 candidates on tuning seeds 1001–1002 at medium demand, high demand and 0.9 times capacity (the car weight stays 1, since Eq. (3.4) compares ratios). The best candidate (motorcycle 1.89, auto-rickshaw 3.02, bus 1.67, truck 0.93, emergency 1.91, queued/moving ratio 2.94, θ = 1.37) was compared with the hand-set weights and a plain count on held-out seeds 1–10. On the held-out seeds the calibrated score changed

delay by -2.0% (medium, $p = 0.374$), -4.9% (high, $p = 0.036$), +2.2% (0.9x capacity, $p = 0.014$) relative to the hand-set weights, whereas a plain vehicle count was worse by +23.2%, +22.7%, +13.4% respectively. The plain count remained clearly worse, confirming the ablation. The parameters are fitted to this simulator and must be re-fitted on field data before deployment; the search is a reproducible procedure, not a learned controller.

5.3 Discussion

Two results qualify the usual claim that density-based control beats fixed timing. Below capacity, the gain over an untuned 20 s plan is not specific to density scoring: a Webster plan, actuated control and max-pressure control achieve the same. Near capacity, density-only switching is worse than fixed timing, because it ends greens while the served queue is still discharging. With the discharge hold, the controller matches the best strategies below capacity and comes close to the highest capacity measured (Webster's, +11.6%), while its rule fits in one line, its red time is bounded and its safety is audited at every step. Its results are, however, almost identical to plain actuated control, which uses no density score: the gap-out test, not the score, does most of the work, and the score's remaining role is to decide when a cleared green hands over. Resuming after preemption with the higher-scoring direction is a direct improvement to test.

The research questions of Section 8.3 are answered as follows: variable greens keep safe change intervals (no violation in 3,524 runs); preemption serves emergency vehicles without delay but costs other traffic more under adaptive control than under fixed timing; near and above capacity the original rule fails and the hold corrects it; and much of the gain below capacity comes from replacing an untuned plan, not from the density score.

5.4 Comparison with Existing Work

Table 5.9 compares the proposed controller with re-implementations of the strategies of existing work, which share its phase sequence, change intervals and green limits. The original papers could not be compared directly, since they use different simulators, demands and metrics.

Table 5.9: Comparison with the strategies of existing work (re-implemented in the same simulator)

Method	**Delay, high demand (s)**	**Capacity vs fixed**	**Source year**
Fixed 20 s plan	7.36 ± 0.22	—	Baseline
Webster-optimized plan [2]	4.60 ± 0.29	+11.6%	1958
Vehicle-actuated [23]	4.63 ± 0.17	+9.6%	2008
Max-pressure [11]	4.62 ± 0.18	−18.8%	2013
Three-level density control [1]	9.49 ± 0.15	+6.3%	2025
Adaptive (density score)	4.76 ± 0.12	−18.9%	**2026**
Adaptive + discharge hold (ours)	4.89 ± 0.23	+9.8%	**2026**

All methods share the phase sequence, change intervals and 8–45 s green limits; delay from Table 5.1, capacity from Table 5.2.

5.5 Limitations

- **Simulation only:** no turning movements, lane changes, pedestrians or driver variability, and no calibration against field data, so absolute delays are not field predictions.
- **Kinematics:** the main model has no reaction time, so queues discharge too fast; the calibrated model matches the saturation headway but lacks start-up delay, so real short greens would cost more and the original rule's capacity loss is if anything understated. The hold's zone must suit local spacing (10 m for the default model, 20.0 m for the calibrated one).
- **Arrivals and baselines:** arrivals are deterministic headways at a re-drawn rate, not Poisson, and emergency vehicles are far more frequent than in reality; the baselines are re-implementations with ideal sensing,

and Webster knows the true demand.

- **Sensing and hardware:** control uses exact states and perfect emergency detection; the vision pipeline was tested only on synthetic frames and is too weak to drive control. A Raspberry Pi 4 deployment (roughly INR 6,000–8,000 in components, detection estimated 5–10 times slower than the laptop) is an estimate to be verified on actual hardware.

Chapter 6

Societal Impact and SDG Alignment

6.1 Primary SDG Relevance

This project directly addresses **SDG 11: Sustainable Cities and Communities**, specifically Target 11.2, which aims to provide access to safe, affordable, accessible and sustainable transport systems for all, and, through reduced congestion, Target 11.6 on the adverse environmental impact of cities.

The project contributes to this goal by showing, in a reproducible simulation, how a low-cost density-aware signal controller behaves: below capacity it shortens delay by 35.3–36.1% relative to a 20 s fixed plan, and above capacity its discharge-hold rule avoids the capacity loss of naive density switching and keeps queues and spillback below those of fixed timing. It also gives emergency vehicles priority while keeping full yellow and all-red intervals, and it identifies honestly where a simple detector would make matters worse. These are simulation results; the real-world benefit depends on field validation.

6.2 Secondary SDG Linkages

Beyond the primary SDG, this project indirectly contributes to:

- **SDG 3 (Good Health and Well-being), Target 3.6:** fewer road injuries is supported by a controller that never shortens change intervals (no violation in 3,524 audited runs) and by emergency-vehicle preemption that reduces ambulance delay at the junction, which can shorten response times.
- **SDG 9 (Industry, Innovation and Infrastructure), Target 9.1:** the system is a low-cost, open-source approach to upgrading existing infras-

tructure: the optional hardware bridge costs an estimated INR 6,000–8,000 in components (an estimate, not yet built).

- **SDG 13 (Climate Action):** shorter queues and fewer stops reduce idling time and therefore fuel use and emissions. The project does not measure emissions; this is an expected, unquantified effect.

6.3 Stakeholder Impact Analysis

Table 6.1: Stakeholder Impact Matrix

Stakeholder Group	**Type**	**Nature of Impact**
Commuters and drivers (cars, motorcycles, auto-rickshaws, buses, trucks)	Primary	Shorter waiting at junctions; fewer stops
Emergency services (ambulance, fire)	Primary	Faster, safe passage through junctions through pre-emption
Municipal and traffic authorities	Secondary	Low-cost, transparent controller with bounded red time and auditable safety; reproducible evaluation method
Researchers and students	Secondary	Open-source simulator, baselines and experiment scripts that regenerate every table
Residents and the environment	Tertiary	Less idling and congestion near junctions (expected, not measured)

6.4 Ethical Considerations

Data privacy. The project uses no real camera footage and no personal data: frames are rendered from the simulator and the ground truth is exact. A real deployment with cameras must count vehicles without storing images or identifying plates or people, and comply with applicable privacy rules. **Bias.** The density weights are heuristic and favour larger vehicles and emergency vehicles by design; they were tested, and calibrated on held-out seeds, but only in simulation, and should be reviewed against local passenger-car-unit factors so that no road user class is systematically disadvantaged. **Safety.**

Green times are bounded, change intervals are never shortened and the signal audit checks each step, but the controller has not been tested on a real junction and must not be deployed on live roads without independent safety validation and a fail-safe fallback to fixed timing. **Misuse.** Emergency preemption could be abused if the emergency flag can be spoofed; a real system needs authenticated vehicle-to-infrastructure messages and rate limits. **Honesty of claims.** Results are reported as simulation results, negative findings (the weak vision pipeline, the capacity loss of the original rule, the equal performance of simpler baselines) are stated plainly, and the use of an AI assistant in development and writing is disclosed.

Chapter 7

Universal Human Values and Harmony with Nature

7.1 Impact on the Material Order

The project uses little material resource: the system runs as software on an ordinary computer, and the optional signal bridge needs a Raspberry Pi or ESP32 board, twelve LEDs with resistors, and (for larger lamps) relay modules. Its purpose is to reduce a material waste, the fuel burnt by vehicles idling in queues at a signal, by shortening waiting time (35.3–36.1% lower delay below capacity in simulation, and shorter queues near capacity with the discharge hold). The effect on energy use is expected but was not measured. Because the controller is software that can run on an existing signal's hardware, little electronic waste is generated; the main lifecycle cost is the energy of the board and of any cameras.

7.2 Impact on the Plant / Bio Order

The project does not act on vegetation directly. Indirectly, shorter queues and less idling mean lower exhaust emissions near junctions, which benefits roadside plants and urban green space. This is an expected, unquantified benefit; the project does not model emissions.

7.3 Impact on the Animal Order

The project has no direct effect on animal habitats. Smoother traffic flow and less idling reduce local air pollution and noise from stop-and-go traffic, which benefits urban wildlife and domestic animals, and drivers who are not

stopped for long are less likely to take risks near the junction. As above, these effects are indirect and were not measured.

7.4 Impact on the Human Order

The human benefits are the central aim. Commuters spend less time waiting; emergency vehicles cross the junction with priority (their stopped delay falls to the start-up component of the metric in every controller) while all other road users are protected by full yellow and all-red intervals that are never shortened (no violation in 3,524 audited runs); and the bounded red time of 50 s means that no approach is starved. The design is transparent: a one-line rule, not a black box, so that authorities and citizens can understand and question it. The project is offered as an open, reproducible resource that students and municipalities can reuse, supporting access to knowledge and fair treatment of all road-user classes. Its honest reporting of limitations supports trust: it states that the gain below capacity is matched by simpler strategies and that the present vision detector cannot yet be trusted to drive the signal.

7.5 Overall Mutual Fulfilment

The project aims at a win-win outcome across all four orders. For people, it shortens waiting, protects safety and gives emergency vehicles priority; for the material order it reduces fuel wasted in idling at little hardware cost; and for the plant and animal orders it lowers, indirectly, emissions and noise at junctions. These benefits reinforce one another rather than trade off: the same shorter queues serve commuters, emergency responders and the environment. The solution is synergistic with the well-being of the material, plant and animal orders and does not exploit them. Care is needed that efficiency does not override safety or fairness, which is why change intervals are protected, red time is bounded, and the claims of the report are limited to what the simulation shows.

Chapter 8

Conclusion and Future Scope

8.1 Summary

This project addressed the problem that low-cost density-based signal controllers are rarely evaluated against strong baselines, near and above capacity, or for signal safety. The following was accomplished:

- Designed and developed a complete simulation-first traffic management system: a microscopic simulator, a density-aware adaptive controller with a discharge hold, an emergency-preemption layer, a Haar-cascade and ByteTrack-style vision pipeline, a FastAPI backend with SQLite persistence, a React dashboard, a hardware bridge, Docker packaging and CI with 119 tests (Chapters 3 and 4).
- Achieved a reduction of mean stopped delay below capacity of 35.3–36.1% relative to a 20 s fixed plan, and with the discharge hold restored and exceeded the fixed plan's capacity (+9.8%), as shown in the results chapter (Chapter 5).
- Demonstrated alignment with SDG 11, with secondary links to SDG 3, 9 and 13 (Chapter 6).

8.2 Conclusions

Based on the experimental results and analysis, the following conclusions are drawn:

1. Conclusion 1: Below capacity, the density-based controller reduces delay by 35.3–36.1% relative to a 20 s fixed plan, but a Webster-optimized plan, actuated control and max-pressure control do as well; most of the gain comes from replacing an untuned plan, not from the density score.

2. Conclusion 2: Near capacity, density-only switching ended greens too early and lost 18.9% of capacity. A discharge-hold rule restored it (+9.8% relative to fixed timing), lowered delay from 0.6 to 1.5 times capacity (not significantly at 1.0×) and kept queues and spillback below those of fixed timing in a 60-minute peak.

3. Conclusion 3: The queued/moving distinction, not the class weights, accounts for the benefit of the density score; data-driven calibration gave only modest, condition-dependent gains.

4. Conclusion 4: Signal safety held in every run (no conflicting green or shortened change interval in 3,524 runs). The findings hold with kinematics calibrated to field saturation headways once the hold's zone is re-tuned (20.0 m).

5. Conclusion 5: The present Haar-cascade detector (4.1% precision, 8.2% recall on synthetic frames) is too weak to drive control: with it, every vehicle-reading controller would be worse than fixed timing near capacity.

8.3 Original Contributions

The original contributions of this project are:

1. A class-weighted, queue-aware density score with a bounded green-extension controller that never shortens yellow or all-red intervals and guarantees a maximum red time, a discharge-hold extension that removes the capacity loss of density-only switching, and an emergency-preemption layer that wraps any phase-based controller and keeps full change intervals, with a per-step signal audit.

2. An empirical evaluation demonstrating, on paired seeded runs, how the controller compares with four re-implemented strategies below, at and above capacity, in a peak period, under calibrated kinematics and under

imperfect sensing, including an ablation and a held-out calibration of the score.

3. A publicly available codebase at `https://github.com/shivam-G05/Smart-Traffic-Management`, in which every table can be regenerated by a script, and a revised research paper reporting the results.

8.4 Limitations

Results are from simulation only: the model has no turning movements, lane changes, pedestrians or driver variability, and arrivals are not Poisson. The main kinematic model has no driver reaction time (a calibrated variant is also reported), the baselines are re-implementations with ideal sensing, the vision pipeline was tested only on synthetic frames and is too weak to drive control, and the hardware bridge has not been built and tested on a physical board. The limitations were managed by calibrating the kinematics against the Highway Capacity Manual, by repeating the comparisons under the calibrated model, by measuring the effect of imperfect sensing, and by labelling every estimate as an estimate.

8.5 Future Scope

Future work can extend this project in the following directions:

- **Short-term:** improve resumption after preemption (return to the higher-scoring direction); add Poisson arrivals and turning movements; train a class-aware detector that suppresses static false positives near the stop line.
- **Medium-term:** close the loop between detection and control with that detector on real footage; test the controller in an established simulator such as SUMO [14]; build and verify the Raspberry Pi/ESP32 prototype and its timing on actual hardware; calibrate the class weights against local passenger-car-unit factors and field data.

- **Long-term:** pilot at a real junction with independent safety validation; extend to coordinated multi-junction corridors; study reinforcement-learning controllers with the same safety audit; broader deployment with municipal authorities.

Bibliography

[1] A. G. Sims and K. W. Dobinson, "The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits," *IEEE Trans. Veh. Technol.*, vol. 29, no. 2, pp. 130–137, May 1980.

[2] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and M. C. Royle, "The SCOOT on-line traffic signal optimisation technique," *Traffic Eng. Control*, vol. 23, no. 4, pp. 190–192, 1982.

[3] K. Nsofwa and S. Tembo, "Density based traffic control system," *Int. J. Traffic Transp. Eng.*, vol. 14, no. 1, pp. 1–8, 2025.

[4] K. Pardeshi, P. Bable, M. Bhalerao, and M. Desai, "Adaptive traffic signal control system using density-based prioritization algorithm on ESP 32 platform," *Int. J. Eng. Res. Technol.*, vol. 15, no. 5, May 2026.

[5] V. B. Kamble, O. N. Mundhe, C. M. Walunjkar, and G. A. Kale, "AI-driven smart traffic management system," *Int. J. Multidiscip. Sci. Manage.*, vol. 2, no. 2, pp. 66–72, 2025.

[6] Z. Hu, W. H. K. Lam, S. C. Wong, A. H. F. Chow, and W. Ma, "Turning traffic surveillance cameras into intelligent sensors for traffic density estimation," *Complex Intell. Syst.*, vol. 9, no. 6, pp. 7171–7195, Dec. 2023.

[7] S. V. Patil, S. Kureshi, and A. Gaikwad, "Vehicle density based traffic signal control system," *Int. J. Novel Res. Develop.*, vol. 9, no. 5, pp. g284–g288, May 2024.

[8] C. M. Thomas, A. B. Nair, A. E. Varghese, A. B. Kumar, and A. P. Seena, "Adaptive traffic signal control using real-time vehicle density," in *AIP Conf. Proc.*, vol. 3260, 2025, p. 020031.

[9] F. V. Webster, "Traffic signal settings," Road Research Laboratory, London, U.K., Road Res. Tech. Paper 39, 1958.

[10] P. Koonce *et al.*, "Traffic signal timing manual," Federal Highway Administration, Washington, DC, USA, Tech. Rep. FHWA-HOP-08-024, 2008.

[11] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transp. Res. C, Emerg. Technol.*, vol. 36, pp. 177–195, Nov. 2013.

[12] P. G. Gipps, "A behavioural car-following model for computer simulation," *Transp. Res. B, Methodol.*, vol. 15, no. 2, pp. 105–111, Apr. 1981.

[13] S. Krauß, "Microscopic modeling of traffic flow: Investigation of collision free vehicle dynamics," Ph.D. dissertation, Univ. of Cologne, Cologne, Germany, 1998.

[14] P. A. Lopez *et al.*, "Microscopic traffic simulation using SUMO," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Maui, HI, USA, 2018, pp. 2575–2582.

[15] *Highway Capacity Manual 2010*. Washington, DC, USA: Transportation Research Board, 2010.

[16] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Kauai, HI, USA, 2001, pp. I–511–I–518.

[17] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[18] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, Mar. 1955.

[19] Y. Zhang *et al.*, "ByteTrack: Multi-object tracking by associating every detection box," in *Computer Vision – ECCV 2022*. Cham, Switzerland: Springer, 2022, pp. 1–21.

[20] H. Wei, G. Zheng, V. Gayah, and Z. Li, "Recent advances in reinforcement learning for traffic signal control," *ACM SIGKDD Explor. Newsl.*, vol. 22, no. 2, pp. 12–18, Jan. 2021.

[21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[22] O. Alzamzami, Z. Alsaggaf, R. AlMalki, R. Alghamdi, A. Babour, and L. Al Khuzayem, "Passable: An intelligent traffic light system with integrated incident detection and vehicle alerting," *Sensors*, vol. 25, no. 18, p. 5760, Sep. 2025.

[23] T. G. Phakathi, M. Sumbwanyambe, and T. S. Hlalele, "Designing intelligent traffic light system with the application of artificial intelligence," in *Proc. Int. Conf. Artif. Intell., Big Data, Comput. Data Commun. Syst.*, 2025, pp. 1–9.

[24] M. Tolani *et al.*, "Machine learning based adaptive traffic prediction and control using edge impulse platform," *Sci. Rep.*, vol. 15, p. 17161, May 2025.

[25] K. Nellore and G. P. Hancke, "Traffic management for emergency vehicle priority based on visual sensing," *Sensors*, vol. 16, no. 11, p. 1892, Nov. 2016.

Appendix A

SDG Compliance Certificate

Certificate of Compliance with United Nations Sustainable Development Goals

This is to certify that the project titled **Smart Traffic Control System Using Vehicle Density Analysis** submitted by the following final year students of the Bachelor of Technology in Information Technology at Ajay Kumar Garg Engineering College, Ghaziabad has been reviewed and found to be in alignment with the UN Sustainable Development Goals (SDGs) listed below. All efforts have been made to ensure no other SDGs are compromised or negatively impacted.

No.	SDG Name	✓
1	No Poverty	☐
2	Zero Hunger	☐
3	Good Health & Well-being	✓
4	Quality Education	☐
5	Gender Equality	☐
6	Clean Water and Sanitation	☐
7	Affordable and Clean Energy	☐
8	Decent Work & Economic Growth	☐
9	Industry, Innovation & Infra.	✓

No.	SDG Name	✓
10	Reduced Inequalities	☐
11	Sustainable Cities & Comm.	✓
12	Responsible Consumption	☐
13	Climate Action	✓
14	Life Below Water	☐
15	Life on Land	☐
16	Peace, Justice & Institutions	☐
17	Partnerships for the Goals	☐

Signatures of Students

Madhav Goel .....

Akash Varshney .....

Abhisekh Chaudhary .....

Shivam Goel .....

Date: October 27, 2026

Signature of Supervisor

Ruchi Gupta

Department of Information Technology

AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD

Date: October 27, 2026

Appendix B

Project Snapshots and Description

B.1 Module 1: Density Score (`decision_engine/density.py`)

Description: The class-weighted, queue-aware density score of Eq. (3.3): every vehicle contributes its class weight times a multiplier that is 1.5 if the vehicle is queued and 0.5 if it is moving. Default weights are in `DENSITY_WEIGHTS` (car 1.0, motorcycle 0.5, auto-rickshaw 0.8, bus 3.0, truck 2.5, ambulance and fire truck 6.0).

Listing B.1: Lane density score

```
def lane_density_score(
    lane: Lane,
    weights: Mapping[VehicleType, float] = DENSITY_WEIGHTS,
    queued_multiplier: float = QUEUED_MULTIPLIER,
    moving_multiplier: float = MOVING_MULTIPLIER,
) -> float:
    score = 0.0
    for vehicle in lane.vehicles:
        weight = weights[vehicle.vehicle_type]
        score += weight * (queued_multiplier if Lane._is_queued(
            vehicle) else moving_multiplier)
    return score
```

B.2 Module 2: Adaptive Controller with Discharge Hold (`decision_engine/adaptive_controller.py`)

Description: The green-end decision of Eqs. (3.4) and (3.6). A green never ends before the minimum or runs past the maximum; between them it may end only when the served direction no longer out-scores the waiting one by the threshold and, with the hold, has gapped out.

Listing B.2: Green-end decision

```
def _should_end_phase(self, intersection: Intersection) -> bool:
    if self.phase not in _GREEN_GROUP:
        return self._elapsed_in_phase >= _FIXED_DURATIONS[self.
            phase]

```

```
    if self._elapsed_in_phase >= self.max_green_s:
        return True
    if self._elapsed_in_phase < self.min_green_s:
        return False
    return self._green_ready_to_end(intersection)

def _green_ready_to_end(self, intersection: Intersection) -> bool:
    if self.hold_while_discharging and self._still_discharging(
       intersection, _GREEN_GROUP[self.phase]):
        return False
    active_score = self._group_density(intersection, _GREEN_GROUP[
       self.phase])
    waiting_score = self._group_density(intersection,
       _WAITING_GROUP[self.phase])
    return active_score <= waiting_score * self.extension_threshold

def _still_discharging(self, intersection: Intersection, group) ->
   bool:
    zone_start = STOP_LINE_POS_M - self.discharge_zone_m
    return any(
        zone_start <= vehicle.position_m < STOP_LINE_POS_M
        for name in group
        for lane in intersection.roads[name].lanes
        for vehicle in lane.vehicles
    )
```

B.3 Module 3: Database Models (`database/models.py`)

Description: The two-table snapshot split (a snapshot and one row per road) used by the analytics collector, as in Table 4.2.

Listing B.3: Snapshot tables

```
class Snapshot(Base):
    __tablename__ = "snapshots"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    run_id: Mapped[int] = mapped_column(ForeignKey("simulation_runs
       .id"))
    sim_time: Mapped[float] = mapped_column(Float)
    captured_at: Mapped[datetime] = mapped_column(DateTime, default
```

```
        =_utcnow)

    run: Mapped["SimulationRun"] = relationship(back_populates="
        snapshots")
    road_snapshots: Mapped[list["RoadSnapshot"]] = relationship(
        back_populates="snapshot")

class RoadSnapshot(Base):
    __tablename__ = "road_snapshots"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    snapshot_id: Mapped[int] = mapped_column(ForeignKey("snapshots.
        id"))
    road_name: Mapped[str] = mapped_column(String(16))
    queue_length: Mapped[int] = mapped_column(Integer)
    vehicle_count: Mapped[int] = mapped_column(Integer)
    density_score: Mapped[float] = mapped_column(Float)
    light_state: Mapped[str] = mapped_column(String(16))
    has_emergency_vehicle: Mapped[bool] = mapped_column(Boolean)
    total_exited: Mapped[int] = mapped_column(Integer)
    total_wait_time_s: Mapped[float] = mapped_column(Float)

    snapshot: Mapped["Snapshot"] = relationship(back_populates="
        road_snapshots")
```

Full Source Code Repository: https://github.com/shivam-G05/Smart-Traffic-Management